

# Baseline架构解析

主要参考：

1. [Gewu-Playground](#)的总动员例程（主要参考 RobotRLAgent.cs ）
  - 原Baseline已经实现 离散步态（walk 或 run） 的能力（两个神经网络）
  - 《人机追逐对抗 0.0》 的目标是用 一个神经网络实现“连续步态切换” 的能力

## 1. 核心控制公式——关键变量

原Baseline控制策略采用 “基于参考轨迹的残差控制（Reference-Guided Residual Control）” 架构。

核心控制公式 (The Final Control Formula)：

$$u_{total} = \underbrace{(k_b \cdot u)}_{\text{RL策略 (大脑)}} + \underbrace{(d_h \cdot u_f + d_0)}_{\text{参考轨迹 (脊髓)}}$$

- **参考轨迹（脊髓）**：由正弦波驱动的基础步态，提供运动的“大趋势”。
- **RL残差（大脑）**：神经网络输出的  $u$ ，用于处理平衡维持和扰动恢复。
- **输入：**
  1. `continuousActions[i]`：来自 RL 策略网络（大脑）的原始输出。
  2. `currentGait`：当前的步态参数（脊髓设定）。
- **输出：**
  - `utotal[]`：**最终关节指令**。融合了 RL 修正量和正弦波参考量，直接发送给 ArticulationBody 的 `xDrive.target`。

### 1.1 参考轨迹相关核心参数—— MoB 的核心

这组参数决定了参考轨迹的“形状”和“风格”，也是实现跑走切换的关键。

其中，更重点关注以下3个超参数：

```
public struct GaitParams
{
    public float T1; // 周期 (Walk~40, Run~25)
    public float dh; // 抬腿幅度 (Walk~10, Run~40)
    public float d0; // 身体姿态偏置 (Walk~0, Run~20)
}
```

- **d0 (Offset / Posture Bias)：**
  - **定义：**正弦波的零点偏置，即关节的**静态平衡位置**。
  - **物理意义：**决定了机器人的**基础姿态**。
  - **示例：**对于 Tinker 机器人，直立状态会导致关节过伸，因此默认需要  $d0=20$ （屈膝 20 度）作为物理零点。
    - **Walk (走)：**  $d0$  较小（接近基础姿态），站姿较高。
    - **Run (跑)：**  $d0$  较大（如下蹲 15 度），降低重心以获得更好的蹬地爆发力。
- **dh (Dynamic Height / Amplitude)：**
  - **定义：**正弦波的振幅。
  - **物理意义：**决定了腿部的**运动幅度**（抬腿多高、跨步多大）。
  - **示例：**  $dh=20$  为标准步幅，  $dh=35$  为高抬腿大跨步。
- **uf1 / uf2 (Unit Factors)：**
  - **定义：**归一化后的正弦波信号，值域  $[0, 1]$ 。
  - **计算公式：**  $uf = \frac{-\cos(2\pi \cdot \frac{tp}{T}) + 1}{2}$

- **作用：**将线性的时间  $tp$  转化为平滑的钟形曲线，驱动关节柔和运动。
- **T1 (Period / Cycle Duration)：**
  - **定义：**半个步态周期所包含的物理帧数。
  - **物理意义：**频率的倒数。T1 越小，动作越快。
  - **数值范围：**
    - T1 = 20：极速跑（每 20 帧迈一步）。
    - T1 = 40：悠闲慢走（每 40 帧迈一步）。
- **tp (Time Phase Counter)：**
  - **定义：**当前的相位计数器。
  - **逻辑：**在 FixedUpdate 中每帧 +1。
    - 当  $0 < tp \leq T1$ ：处于 **左腿驱动相**（左腿动，右腿支撑）。
    - 当  $T1 < tp \leq 2 \cdot T1$ ：处于 **右腿驱动相**（右腿动，左腿支撑）。
    - 当  $tp > 2 \cdot T1$ ：重置为 0，开始新循环。

代码逻辑上确实没有“腾空相”，但物理结果上会出现“腾空”。

**控制信号 (Signal) 角度——确实没有腾空相。**

- $0 \sim T1$ ：左腿有信号。
- $T1 \sim 2 \cdot T1$ ：右腿有信号。
- **信号是无缝衔接的。**代码中并没有一段  $T_{flight}$  时间让  $uf1$  和  $uf2$  同时为 0。

**动力学角度——动力学涌现 (Emergent Dynamics)，因惯性而腾空。**

当 T1 很小（频率快）且 dh 很大（蹬地猛）时：

1. **蓄力：**正弦波前半段，腿弯曲。
2. **爆发：**正弦波后半段，腿猛烈伸直（蹬地）。
3. **起飞：**由于蹬地的**冲量 (Impulse)** 非常大，产生的垂直速度  $V_y$  足以让机器人在地心引力把它拉下来之前，飞在空中。
4. **滞空：**在空中时，虽然代码已经切换到了“右腿相”，并且右腿可能已经在尝试往下踩了，但因为身体飞得太高，脚够不着地——**这就是腾空相。**

**比喻：**

这好比你骑自行车。你的脚是一圈一圈不停蹬的（信号连续），但如果你骑得足够快并冲过一个小坡，车子就会腾空（物理结果）。你不需要停止蹬车才能腾空。

**为什么 Run 需要 d0 (蹲姿)？**

这也解释了为什么 Run 模式下 d0 要设为 15（蹲下）：

- 如果你站得很直（ $d0=0$ ），腿已经伸直了，就没有“伸长空间”来蹬地了。
- 只有先蹲下（ $d0=15$ ），像弹簧一样压缩，当正弦波让腿伸直时，才能输出爆发力，从而**制造出代码里没有写、但物理上存在的腾空相。**

## 1.2 RL策略相关参数

$$u_{total-RL}[i] = kb[i] * u[i] + kb1[i] * ut[i] + kb2[i] * utt[i]$$

- **u[]：**RL Residual Action / 神经网络残差动作。**对应传统PID控制中的P项**
  - **定义：**由强化学习策略网络 (Policy Network) 根据当前观测实时计算出的原始控制信号。
  - **物理本质：**它是对基础正弦波轨迹的**动态修正项**。当基础步态 (Walk/Run) 无法应对复杂地形或扰动时，RL 通过输出非零的  $u$  值，在基础角度上叠加一个微调力矩，从而实现动态平衡。
  - **信号处理：**代码中对  $u$  进行了低通滤波（系数  $kk=0.9$ ），以过滤高频抖动，确保电机输出平滑，模拟生物肌肉的粘滞特性。
- **kb[]：****增益 (Gain)**。用于放大 RL 的输出信号（如设为 30）。
  - **注：**在 Debug 过程中，如果发现 RL“对抗”正弦波（导致跛脚），通常需要降低此值。

- `ut[]`, `utt[]`, `kb1[]`, `kb2[]` : 对应 PID 控制中的一重积分项 (I项, 稳态误差 Steady-state Error), 二重积分项 (II项) 以及对应的增益系数

```
for (int i = 0; i < ActionNum; i++)
{
    u[i] = u[i] * kk + (1 - kk) * continuousActions[i];
    ut[i] += u[i];
    utt[i] += ut[i];
    utotal[i] = kb[i] * u[i] + kb1[i] * ut[i] + kb2[i] * utt[i];
    if (fixbody) utotal[i] = 0;
}
```

### 双足: 忽略 `ut[]`, `utt[]`; 轮式考虑

- 积分项 (I) 擅长通过累积历史偏差来消除稳态误差 (提供对抗恒定阻力的“后劲”);
- 双足 (Biped): 仅用 P项 (`u`)。核心诉求是 极速响应, 动态平衡需要 0.02s 级的瞬时修正, 无法容忍积分项带来的相位滞后, 误差修补全靠 NN 的高频实时计算。
- 轮腿 (LegWheeled): 引入 I/II项 (`ut/utt`)。核心诉求是 稳态维持, 利用积分项消除静差以克服恒定的地面摩擦力, 确保轮子保持平滑的恒定转速。

| 传统PID控制视角            |                      |                     | 动力学视角                                 |
|----------------------|----------------------|---------------------|---------------------------------------|
| 变量/参数                | 控制理论对应               | 数学运算                | 物理作用                                  |
| <code>u</code>       | P (Proportional)     | $f(x)$              | 主要发力。神经网络说“去这里”, 电机就用力往那里转。           |
| <code>ut</code>      | I (Integral)         | $\int f(x)dt$       | 消除误差。如果没这就位, 就持续加力 (双足中禁用)。           |
| <code>utt</code>     | II (Double Integral) | $\int\int f(x)dt^2$ | 位移/惯性维持。极其迟钝 (双足中禁用)。                 |
| <code>damping</code> | D (Derivative)       | $f'(x)$             | 刹车/稳定。防止动作太猛产生震荡。代码中固定为 100, RL 无法控制。 |

低通滤波 (Low-pass Filter) 处理 (平滑处理) —— 这一帧的动作 = 90% 的上一帧动作 + 10% 的新想法  
 在 `OnActionReceived` 中, `u` 并不是直接等于神经网络的输出, 而是经过了一个低通滤波。其中 `continuousActions[i]` 是神经网络的输出

## 2. 全生命周期数据流 (Lifecycle & Data Flow)

Unity-cs主要包含以下组件:

```
public override void Initialize()
public override void OnEpisodeBegin()
public override void CollectObservations(VectorSensor sensor)
public override void OnActionReceived(ActionBuffers actionBuffers)
void FixedUpdate()
```

系统的运行顺序: 1-2-3-2-3-.....

### 1. 启动与初始化 (Initialize)

- 时机: 脚本加载时执行一次。
- 操作: 构建身体 (获取关节引用), 统计自由度 (`ActionNum`), 初始化默认步态参数。

### 2. 场景重置 (OnEpisodeBegin)

- 时机: 训练开始或机器人摔倒/超时。
- 操作:
  1. 物理归位: 重置位置与速度。

2. ~~任务采样~~：调用 ~~RandomizeGaitCommand()~~，随机生成本回合的战术目标，这是 MoB 训练多样性的源头。

### 3. 物理步进循环 (Physics Step Loop)

这是以 50Hz 运行的主循环，单帧数据流向如下：

- **Step A: 脊髓节律 (FixedUpdate 前半部分)**
  - **时钟推进**： $tp++$ 。
  - **波形生成**：根据当前 T1 计算参考轨迹信号  $uf1 / uf2$ 。
  - **参数演化**：(训练时) 对 `GaitParams` 进行平滑插值。
- **Step B: 感知 (CollectObservations)**
  - **操作**：读取物理状态 (速度/角度) + **注入相位** (  $\sin/\cos$  ) 与步态指令 (T1, dh, d0)。
  - **流向**：打包数据，发送给 ML-Agents Python 端。
- **Step C: 大脑决策 (ML-Agents 内部黑盒)**
  - **Input**：Step B 收集的 Observation 向量。
  - **Process**：策略网络 (Policy Network) 前向传播。
  - **Output**：`continuousActions[i]` (原始动作指令)。
- **Step D: 运动混合与下发 (OnActionReceived)**
  - **信号处理**：低通滤波  $u_t = 0.9u_t - 1 + 0.1a_{raw}$   $u_t = 0.9u_t - 1 + 0.1a_{raw}$
  - **指令融合**：执行核心公式  $u_{total} = (kb \cdot u) + (dh \cdot uf + d0)$   $u_{total} = (kb \cdot u) + (dh \cdot uf + d0)$
  - **下发执行**：调用 `SetJointTargetDeg` 将最终指令发送给电机 (`xDrive.target`)。
- **Step E: 物理模拟 (Unity Physics Engine)**
  - **黑盒计算**：物理引擎结合重力、摩擦力、惯性与电机指令，计算出机器人下一帧的**真实状态** (产生腾空等动力学现象)。
- **Step F: 结果评估与反馈 (FixedUpdate 后半部分)**
  - **计算奖励**：基于物理模拟后的真实状态计算 Reward。
  - **闭环反馈**：将 Reward 发回给 RL 算法；(训练模式下) Python 端利用积累的 Reward 和 Observation 序列更新神经网络参数。